



UCCD2203 DATABASE SYSTEMS

Lecture 5

By

Mr. Cheang Kah Wai, Alvin

Department of Information Systems, FICT

Lecture 5 Topic: Transaction Management - Part 1

- Introduction to Transaction Management Concepts and Theory
- Concurrency Control Techniques

Transaction

- A transaction is a logical unit of work on the database.
 - It may be an entire program or a part of a program, or a single SQL statement, and it may involve any number of operations on the database.
-

Transaction Support

- Action, or series of actions (*Read* and *Write*), carried out by user or application, which accesses or changes the contents of database.
- Transforms database from one consistent state to another, although consistency may be violated during transaction.

Example - Transaction

```
read(staffNo = x, salary)
```

```
salary = salary * 1.1
```

```
write(staffNo = x, new_salary)
```

(a)

```
delete(staffNo = x)
```

```
for all PropertyForRent records, pno
```

```
begin
```

```
read(propertyNo = pno, staffNo)
```

```
if (staffNo = x) then
```

```
begin
```

```
staffNo = newStaffNo
```

```
write(propertyNo = pno, staffNo)
```

```
end
```

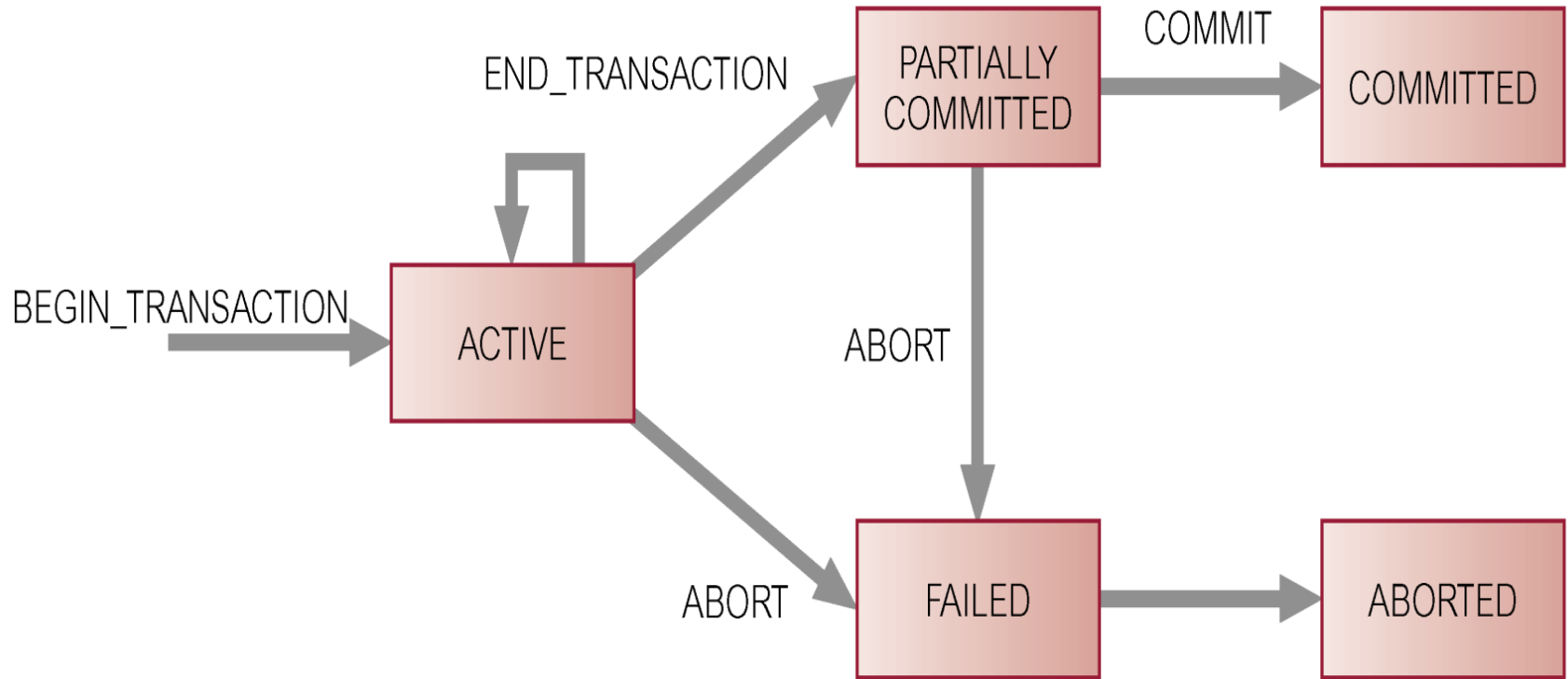
```
end
```

(b)

Transaction Support

- Can have one of two outcomes:
 - **Committed** - transaction *commits* and database reaches a new consistent state.
 - **Aborted** - transaction *aborts*, and database must be restored to consistent state before it started.
 - Such a transaction is *rolled back* or *undone*.
- Committed transaction cannot be aborted.
- Aborted transaction is *rolled back* and can be restarted later.

State Transition Diagram for Transaction



In addition to the obvious states of ACTIVE, COMMITTED and ABORTED, there are other two states:

- PARTIALLY COMMITTED, FAILED

Properties of Transactions

Four basic *Atomicity, Consistency, Isolation, and Durability (ACID) properties* properties of a transaction are:

Atomicity 'All or nothing' property.

Consistency Must transform database from one consistent state to another (*data integrity*).

Isolation Partial effects of incomplete transactions should not be visible to other transactions.

Durability Effects of a committed transaction are permanent and must not be lost because of later failure.

Concurrency Control

- Although two transactions may be correct in themselves, interleaving of operations may produce an incorrect result.
- Process of managing **simultaneous operations** on the database without having them interfere with one another.
- Prevents interference when two or more users are accessing database simultaneously and at least one is updating data.

Need for Concurrency Control

- Three examples of potential **problems caused by concurrency in multiuser environments**:
 - ❑ **Lost update problem**
 - ❑ **Uncommitted dependency problem**
 - ❑ **Inconsistent analysis problem**
- Coordinates **simultaneous transaction execution** in **multiprocessing** database
- Ensure **serializability** of transactions in **multiuser** database environment.

Lost Update Problem

- A successfully **completed update** operation by one user can **be overridden by another user**.

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

Lost Update Problem - example

- T_1 withdrawing \$10 from an account with bal_x , initially \$100.
 - T_2 depositing \$100 into same account.
 - Serially, final balance would be \$190.
-
- Loss of T_2 's update is avoided by preventing T_1 from reading bal_x until after update.

Uncommitted Dependency (or dirty read) Problem

- This occurs when one transaction is allowed **to see** the intermediate results of another transaction **before it has committed**.

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal _x)	100
t ₃		bal _x = bal _x + 100	100
t ₄	begin_transaction	write(bal _x)	200
t ₅	read(bal _x)	:	200
t ₆	bal _x = bal _x - 10	rollback	100
t ₇	write(bal _x)		190
t ₈	commit		190

Uncommitted Dependency Problem

- **T_4 updates bal_x to \$200 but it aborts,** so bal_x should be back at original value of \$100.
- **T_3 has read new value** of bal_x (\$200) and uses value as basis of \$10 reduction, giving a new balance of \$190, instead of \$90.
- Problem is avoided by preventing T_3 from reading bal_x until after T_4 commits or aborts.

Inconsistency analysis problem

- The two problems concentrated on transactions that are updating the database and their interference may corrupt the database.
- However, transactions that only read the database can also produce inaccurate results if they are allowed to read partial results of incomplete transactions that are simultaneously updating the database.

The inconsistency analysis problem

Inconsistent Analysis (*fuzzy read or nonrepeatable read*) Problem

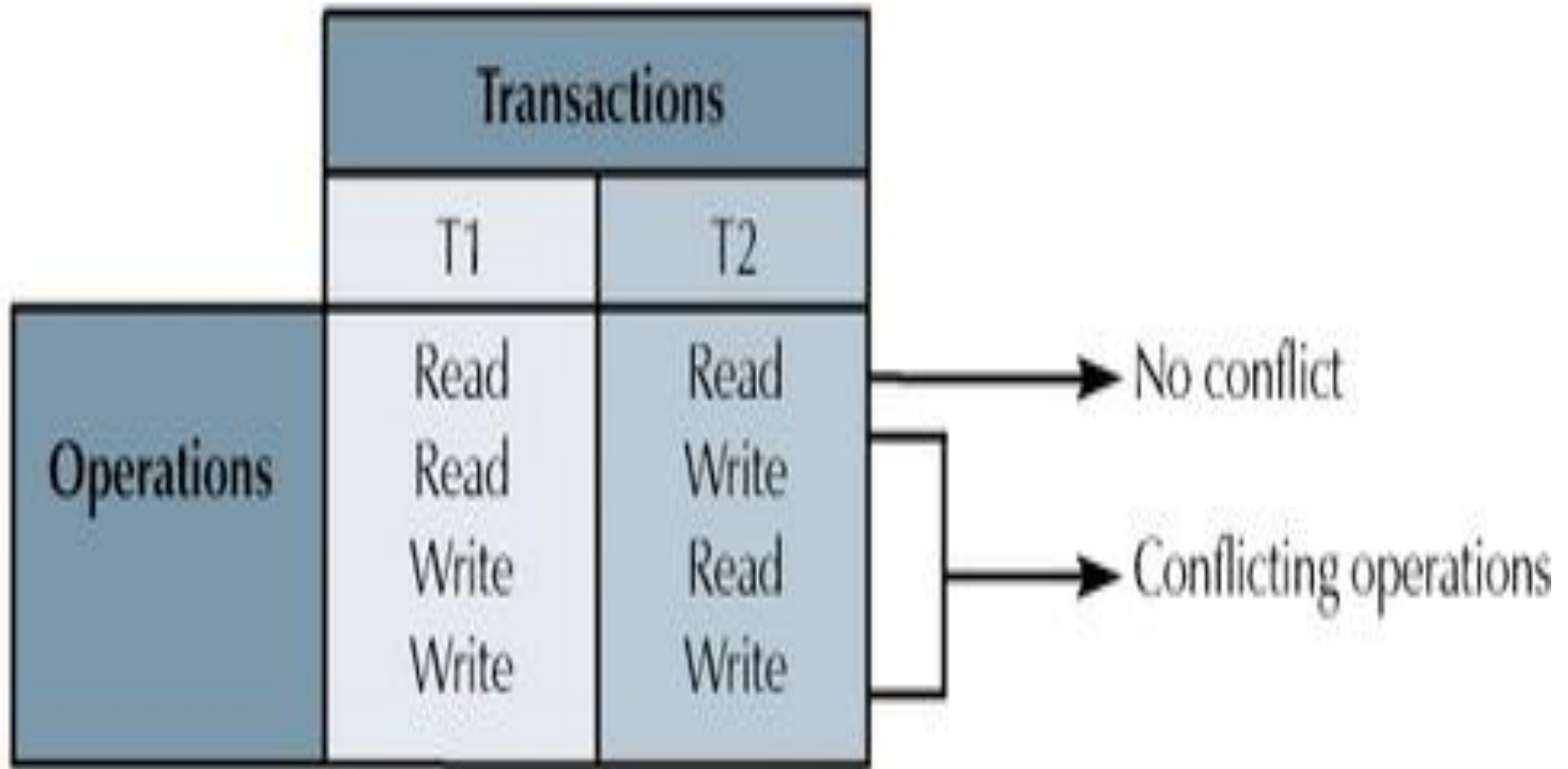
- Occurs when **transaction reads** several values but **second transaction updates** some of them during execution of the first.

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	read(bal _x)	read(bal _x)	100	50	25	0
t ₄	bal _x = bal _x - 10	sum = sum + bal _x	100	50	25	100
t ₅	write(bal _x)	read(bal _y)	90	50	25	100
t ₆	read(bal _z)	sum = sum + bal _y	90	50	25	150
t ₇	bal _z = bal _z + 10		90	50	25	150
t ₈	write(bal _z)		90	50	35	150
t ₉	commit	read(bal _z)	90	50	35	150
t ₁₀		sum = sum + bal _z	90	50	35	185
t ₁₁		commit	90	50	35	185

Inconsistent Analysis Problem

- T_6 is totaling balances of account x (\$100), account y (\$50), and account z (\$25).
- Meantime, T_5 has transferred \$10 from bal_x to bal_z , so T_6 now has wrong result (\$10 too high).
- Problem is avoided by preventing T_6 from reading bal_x and bal_z until after T_5 completed updates.

Read/Write Conflict Scenarios:



Conflicting Database Operations Matrix

Serializability

- Objective of a concurrency control protocol is **to schedule transactions** in such a way as to **avoid any interference**.
- Could **run transactions serially**, but this limits the degree of concurrency or parallelism in system.
- Serializability identifies those executions of transactions guaranteed to ensure **consistency**.

Serializability

Schedule

A sequence of reads/writes by set of concurrent transactions that preserves **the order of the operations** in each of the individual transactions

Serial Schedule

Schedule where operations of each transaction are **executed consecutively** without any interleaved operations from other transactions.

- **No guarantee** that results of all serial executions of a given set of transactions will be identical.

Nonserial Schedule

- Schedule where operations from set of concurrent transactions are interleaved.
- Objective of serializability is to **find nonserial schedules that allow transactions to execute *concurrently*** without interfering with one another.
- In other words, we want to find nonserial schedules that are equivalent to *some* serial schedule. Such a schedule is called ***serializable***.

Serializability

In serializability, ordering of read/writes is important:

- (a) If two transactions only read a data item, they do not conflict and order is not important.
- (b) If two transactions **either read or write** completely *separate* data items, they do not conflict and *order is not important*.
- (c) If one transaction writes a data item and another **reads or writes *same* data item**, ***order of execution is important***.

Conflict Serializability

- *Conflict* serializable schedule orders any conflicting operations in same way as some serial execution.
- Under *constrained write rule* (transaction updates a data item based on its old value, which is first read by the transaction), we can use *precedence graph* to test for serializability.

Precedence Graph

- Create a node for each transaction;
 - a directed edge $T_1 \rightarrow T_2$, if T_2 reads the value of an item written by T_1 ;
 - a directed edge $T_1 \rightarrow T_2$, if T_2 writes a value into an item after it has been read by T_1 .
 - a directed edge $T_1 \rightarrow T_2$, if T_2 writes a value into an item after it has been written by T_1 .
- If precedence graph contains **cycle**, the schedule is ***not* conflict serializable**.

Example - Non-conflict serializable schedule

- T_9 is transferring 100 from one account with balance bal_x to another account with balance bal_y .
- T_{10} is increasing balance of these two accounts by 10%.
- Precedence graph has a cycle and so is not serializable.

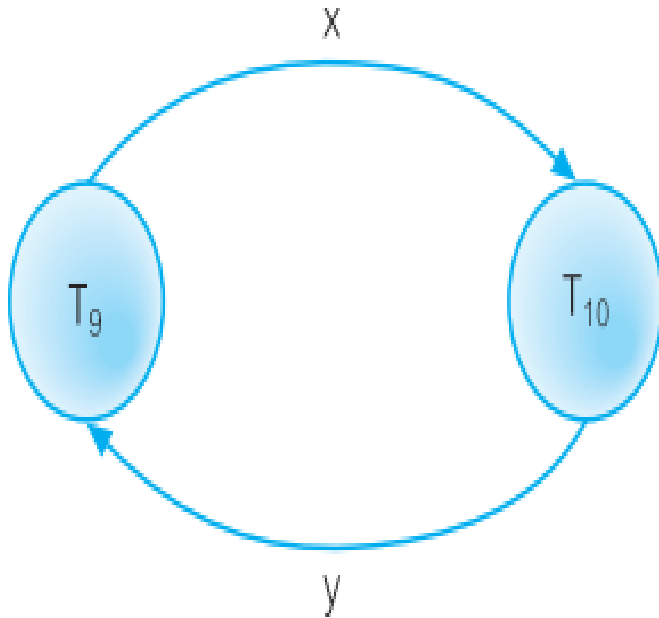
Example - Non-conflict serializable schedule

T_9 is transferring 100 from one account with balance bal_y to another account with balance bal_x .

T_{10} is increasing balance of these two accounts by 10%.

Time	T_9	T_{10}
t_1	begin_transaction	
t_2	read(bal_x)	
t_3	$bal_x = bal_x + 100$	
t_4	write(bal_x)	begin_transaction
t_5		read(bal_x)
t_6		$bal_x = bal_x * 1.1$
t_7		write(bal_x)
t_8		read(bal_y)
t_9		$bal_y = bal_y * 1.1$
t_{10}		write(bal_y)
t_{11}	read(bal_y)	commit
t_{12}	$bal_y = bal_y - 100$	
t_{13}	write(bal_y)	
t_{14}	commit	

Example - Non-conflict serializable schedule



Precedence graph for Transactions T_9 and T_{10} showing a cycle, so schedule is not conflict serializable.

T_9

```
begin_transaction
  read(balx)
  balx = balx + 100
  write(balx)
```

```
  read(baly)
  baly = baly - 100
  write(baly)
commit
```

T_{10}

```
begin_transaction
  read(balx)
  balx = balx * 1.1
  write(balx)
  read(baly)
  baly = baly * 1.1
  write(baly)
commit
```

View Serializability

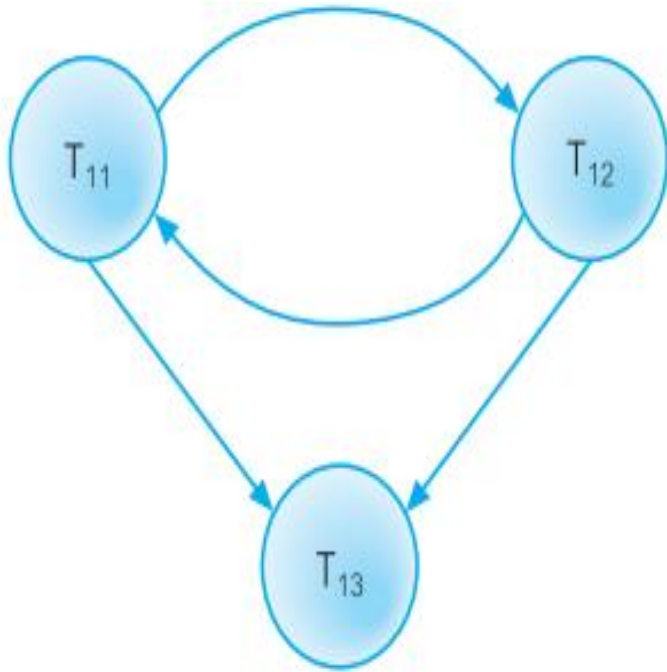
- Offers less stringent definition of schedule equivalence than conflict serializability.
- Two schedules S_1 and S_2 are view equivalent if the following three conditions hold:
 - For each data item x , if T_i reads initial value of x in S_1 , T_i must also read initial value of x in S_2 .
 - For each read on x by T_i in S_1 , if value read by x is written by T_j , T_i must also read value of x produced by T_j in S_2 .
 - For each data item x , if last write on x performed by T_i in S_1 , same transaction must perform final write on x in S_2 .

View Serializability

- Schedule is view serializable if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is view serializable, although converse is not true.
- It can be shown that any view serializable schedule that is not conflict serializable contains one or more *blind writes*.

Example - View Serializable schedule

Not conflict serializable



Precedence graph for
view serializable
schedule.

Time	T_{11}	T_{12}	T_{13}
t_1	begin_transaction		
t_2	read(bal_x)		
t_3		begin_transaction	
t_4		write(bal_x)	
t_5		commit	
t_6	write(bal_x)		
t_7	commit		
t_8			begin_transaction
t_9			write(bal_x)
t_{10}			commit

Concurrency Control Techniques

- Two basic concurrency control techniques:
 - Locking
 - Timestamping
- Both are conservative (**pessimistic**) approaches: delay transactions in case they conflict with other transactions.
- **Optimistic** methods assume conflict is rare and only **check for conflicts at commit**.

Locking

Transaction uses locks to deny access to other transactions and so prevent incorrect updates.

- Most widely used approach to ensure **serializability**.
- Generally, a transaction must claim a *shared* (*read*) or *exclusive* (*write*) lock on a data item before read or write.
- Lock prevents another transaction from modifying item or even reading it, in the case of a **write lock**.

Lock Modes

- Two main lock modes:
 - Binary locks
 - Shared locks/Exclusive locks

Locking - Basic Rules

- If transaction has **shared lock on item**, **can read** but not update item.
- If transaction has **exclusive lock on item**, **can both read and update item**.
- Reads cannot conflict, so more than one transaction can hold **shared locks simultaneously on same item**.
- Exclusive lock gives transaction exclusive access to that item.

Concurrency Control-with Locking Methods

- Lock guarantees current transaction exclusive use of data item
- Acquires lock prior to access
- Lock released when transaction is completed
- DBMS automatically initiates and enforces locking procedures
- Managed by lock manager
- Lock granularity indicates level of lock use – database, table, page, row, or field lock levels

Database-Level Locking Sequence

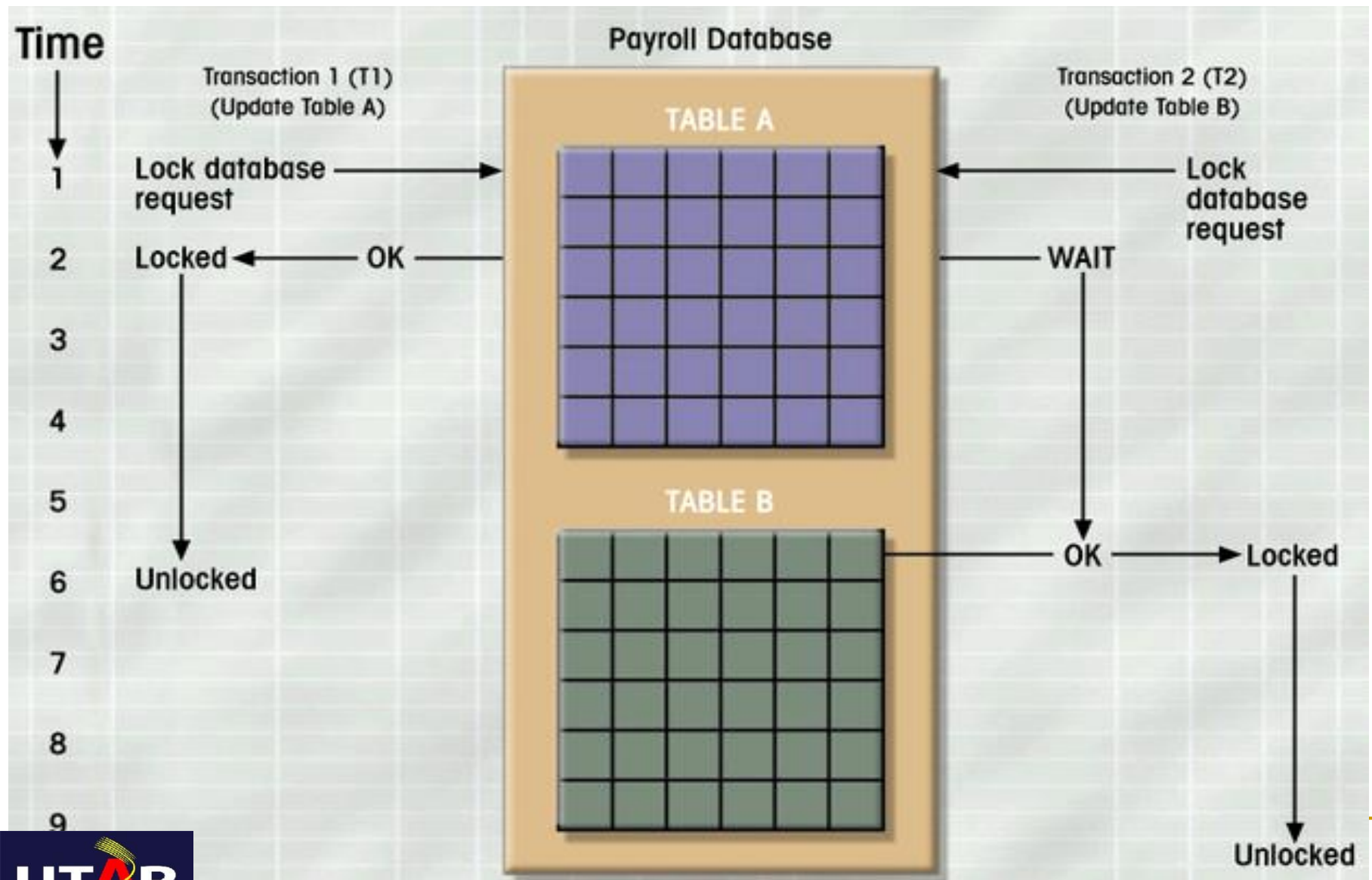
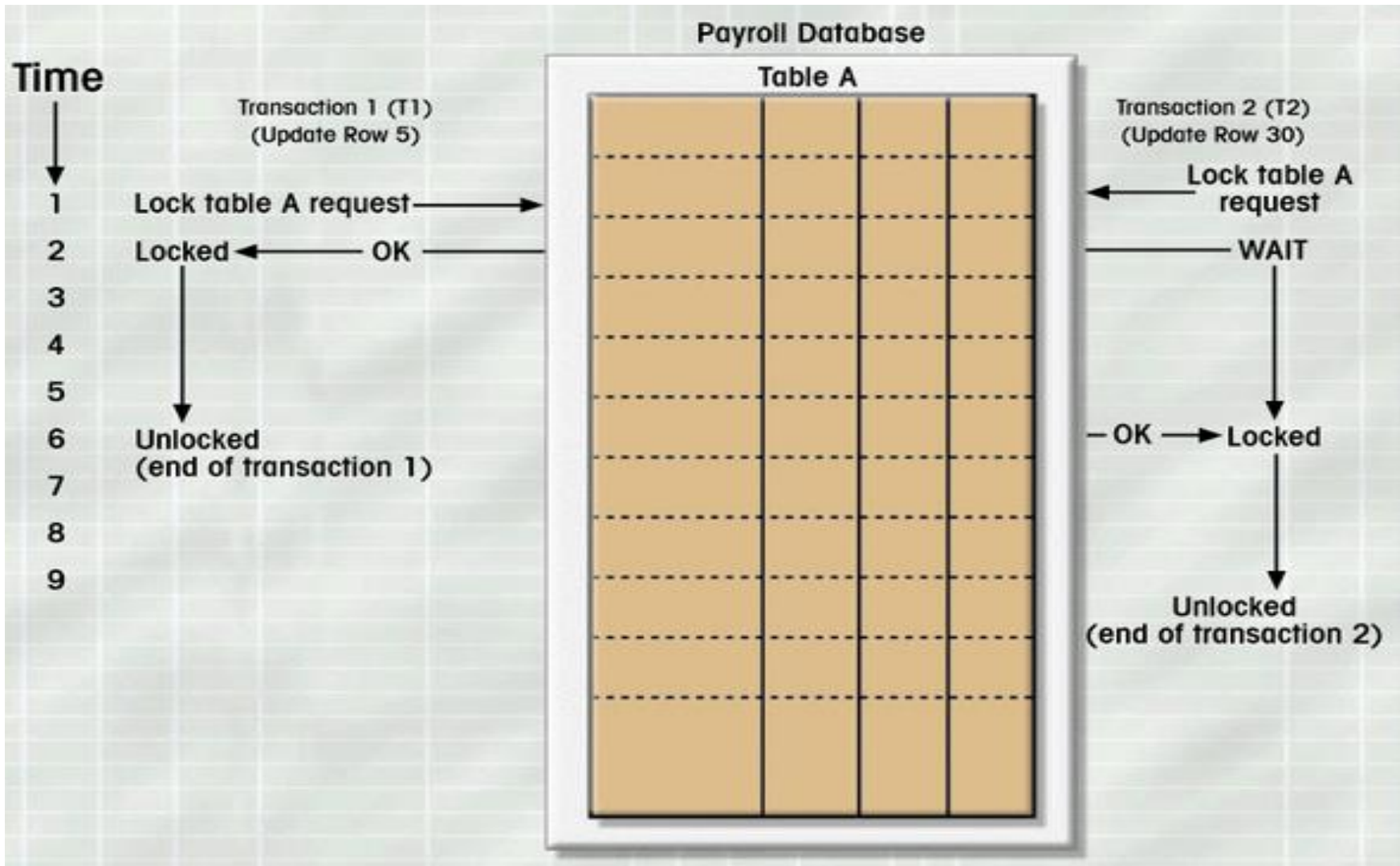


Table-Level Lock



Example - Incorrect Locking Schedule

- For two transactions above, a schedule using these rules is:

$S = \{ \text{write_lock}(T_9, \text{bal}_x), \text{read}(T_9, \text{bal}_x), \text{write}(T_9, \text{bal}_x), \text{unlock}(T_9, \text{bal}_x), \text{write_lock}(T_{10}, \text{bal}_x), \text{read}(T_{10}, \text{bal}_x), \text{write}(T_{10}, \text{bal}_x), \text{unlock}(T_{10}, \text{bal}_x), \text{write_lock}(T_{10}, \text{bal}_y), \text{read}(T_{10}, \text{bal}_y), \text{write}(T_{10}, \text{bal}_y), \text{unlock}(T_{10}, \text{bal}_y), \text{commit}(T_{10}), \text{write_lock}(T_9, \text{bal}_y), \text{read}(T_9, \text{bal}_y), \text{write}(T_9, \text{bal}_y), \text{unlock}(T_9, \text{bal}_y), \text{commit}(T_9) \}$

Time	T ₉	T ₁₀
t ₁	begin_transaction	
t ₂	read(bal_x)	
t ₃	bal_x = bal_x + 100	
t ₄	write(bal_x)	begin_transaction
t ₅		read(bal_x)
t ₆		bal_x = bal_x * 1.1
t ₇		write(bal_x)
t ₈		read(bal_y)
t ₉		bal_y = bal_y * 1.1
t ₁₀		write(bal_y)
t ₁₁	read(bal_y)	commit
t ₁₂	bal_y = bal_y - 100	
t ₁₃	write(bal_y)	
t ₁₄	commit	

Example - Incorrect Locking Schedule

- If at start, $bal_x = 100$, $bal_y = 400$, result should be:
 - $bal_x = 220$, $bal_y = 330$, if T_9 executes before T_{10} ;
or
 - $bal_x = 210$, $bal_y = 340$, if T_{10} executes before T_9 .
- However, result of executing S gives $bal_x = 220$ and $bal_y = 340$.
- S is not a serializable schedule.
→ Does not guarantee serializability.

Example - Incorrect Locking Schedule

- Problem is that transactions release locks too soon, resulting in loss of total isolation and atomicity.
- To guarantee serializability, need an additional protocol concerning the positioning of lock and unlock operations in every transaction.

Two-Phase Locking (2PL)

Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction.

- Two phases for transaction:
 - **Growing phase** - acquires all locks but cannot release any locks.
 - **Shrinking phase** - releases locks but cannot acquire any new locks.

Lost Update Problem

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

Preventing Lost Update Problem using 2PL

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	write_lock(bal_x)	100
t_3	write_lock(bal_x)	read(bal_x)	100
t_4	WAIT	$bal_x = bal_x + 100$	100
t_5	WAIT	write(bal_x)	200
t_6	WAIT	commit/unlock(bal_x)	200
t_7	read(bal_x)		200
t_8	$bal_x = bal_x - 10$		200
t_9	write(bal_x)		190
t_{10}	commit/unlock(bal_x)		190

Deadlock

An impasse that may result when two (or more) transactions are each **waiting for locks** held by the **other to be released**.

Time	T ₁₇	T ₁₈
t ₁	begin_transaction	
t ₂	write_lock(bal_x)	begin_transaction
t ₃	read(bal_x)	write_lock(bal_y)
t ₄	bal_x = bal_x - 10	read(bal_y)
t ₅	write(bal_x)	bal_y = bal_y + 100
t ₆	write_lock(bal_y)	write(bal_y)
t ₇	WAIT	write_lock(bal_x)
t ₈	WAIT	WAIT
t ₉	WAIT	WAIT
t ₁₀	:	WAIT
t ₁₁	:	:

Deadlock

- Only one way to **break deadlock**: abort one or more of the transactions.
- Deadlock should be transparent to user, so DBMS should restart transaction(s).
- Three general techniques for **handling deadlock**:
 - ❑ **Timeouts.**
 - ❑ **Deadlock prevention.**
 - ❑ **Deadlock detection and recovery.**

Deadlock Prevention

- **DBMS looks ahead to see if transaction would cause deadlock and never allows deadlock to occur.**
- **Could order transactions using transaction timestamps:**
 - Wait-Die - only an **older transaction can wait for younger one, otherwise transaction is aborted (*dies*) and restarted** with same timestamp. Eventually, it will become the oldest active transaction and will not die.

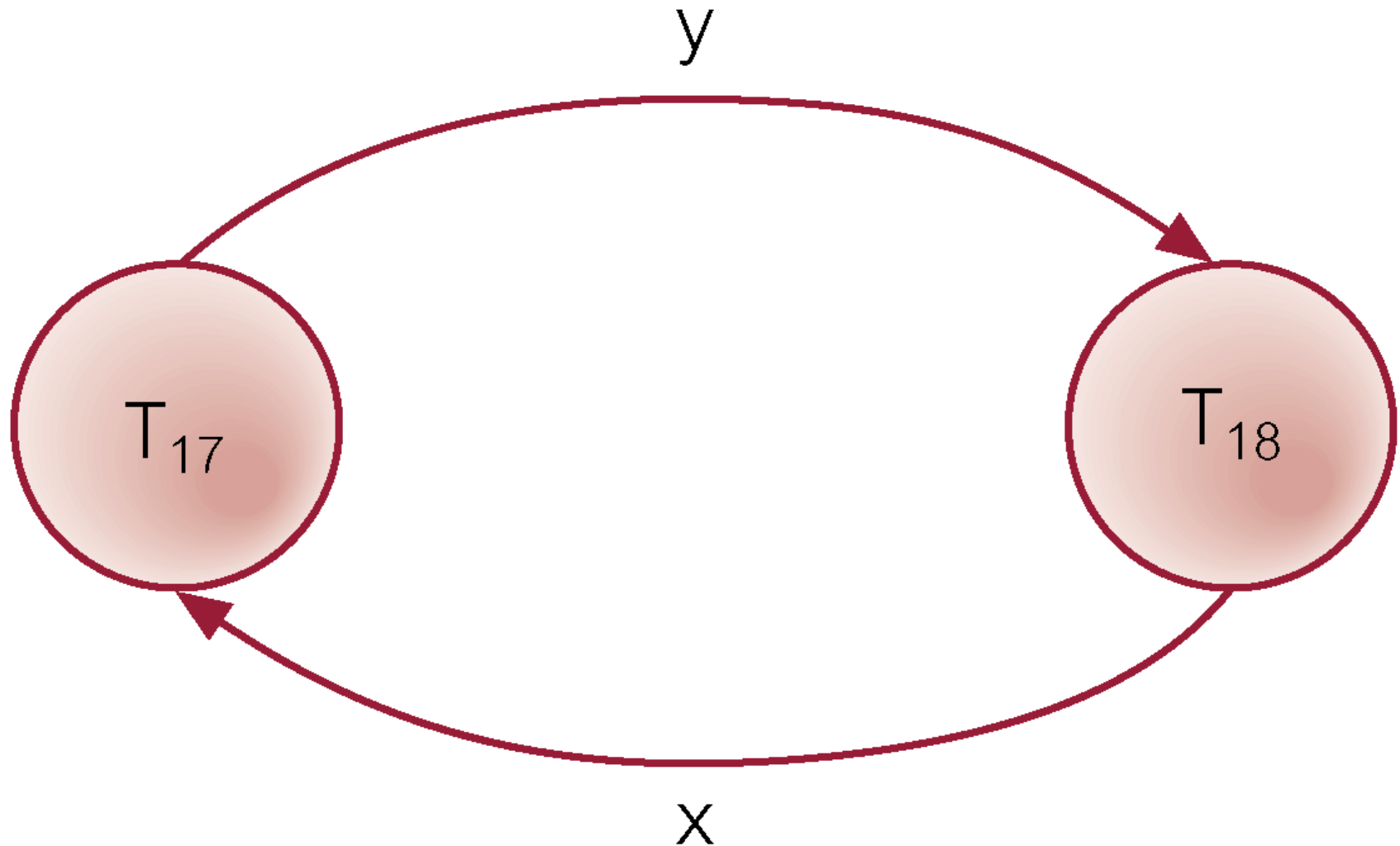
Deadlock Prevention

- ❑ Wound-Wait - only a **younger transaction can wait for an older one**. If older transaction requests lock held by younger one, **younger one is aborted** (*wounded*).

Deadlock Detection and Recovery

- **DBMS allows deadlock to occur** but recognizes it and breaks it.
- Usually handled by construction of **wait-for graph** (WFG) showing transaction dependencies:
 - Create a node for each transaction.
 - Create edge $T_i \rightarrow T_j$, if T_i waiting to lock item locked by T_j .
- **Deadlock exists if and only if WFG contains cycle.**
- WFG is created at regular intervals.

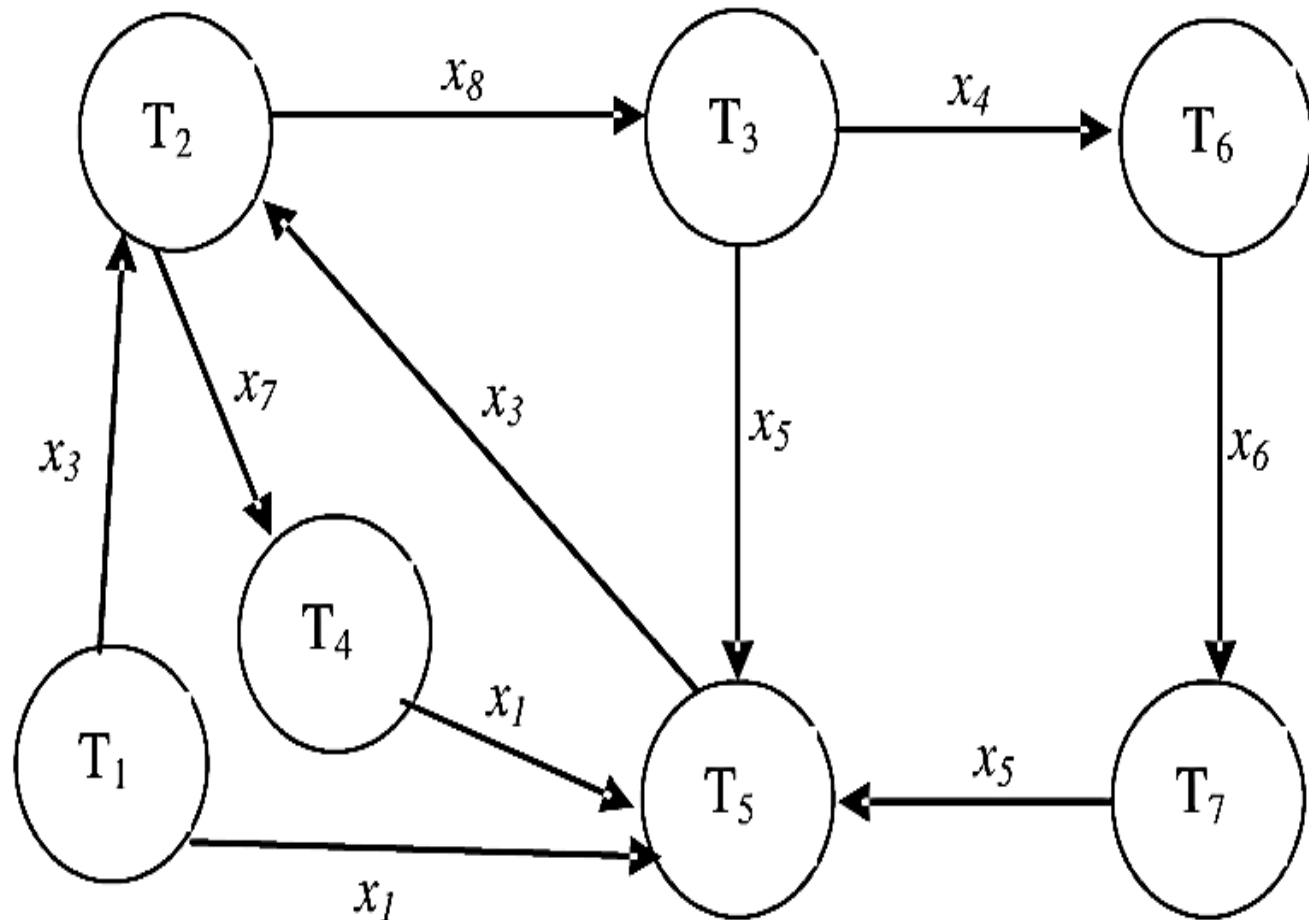
Example - Wait-For-Graph (WFG)



Transaction	Data items locked by transaction	Data items transaction is waiting for
T_1	x_2	x_1, x_3
T_2	x_3, x_{10}	x_7, x_8
T_3	x_8	x_4, x_5
T_4	x_7	x_1
T_5	x_1, x_5	x_3
T_6	x_4, x_9	x_6
T_7	x_6	x_5

Produce a wait-for-graph for the following transaction scenario and determine whether deadlock exists.

- Create a node for each transaction.
- Create edge $T_i \rightarrow T_j$, if T_i waiting to lock item locked by T_j .



Cycles in graph implies that deadlock exists.

(T2 -> T4 -> T5 -> T2), (T2 -> T3 -> T5 -> T2), (T2 -> T3 -> T6 -> T7 -> T5 -> T2)

Timestamping

- Transactions ordered globally so that **older transactions**, transactions with *smaller* timestamps, get *priority* in the event of conflict.
- Conflict is **resolved by rolling back and restarting transaction**.
- No locks so no deadlock.

Timestamping

Timestamp

A unique identifier created by DBMS that indicates relative starting time of a transaction.

- Can be generated by using system clock at time transaction started, or by incrementing a logical counter every time a new transaction starts.

Timestamping

- Read/write proceeds only if *last update on that data item* was carried out by an older transaction.
- Otherwise, transaction requesting read/write is restarted and given a new timestamp.
- Also timestamps for data items:
 - read-timestamp - timestamp of last transaction to read item.
 - write-timestamp - timestamp of last transaction to write item.

The End Of Lecture 5

Thank You